7.3.1. The distributional Bellman equation

The Bellman equation implicitly assumes that the environment is deterministic and thus that observed rewards are deterministic (i.e., the observed reward will be always the same if you take the same action in the same state). In some cases this is true, but in other cases it is not. All the games we have used and will use (except for Gridworld) involve at least some amount of randomness. For example, when we downsample the frames of a game, two originally different states will get mapped into the same downsampled state, leading to some unpredictability in observed rewards.

In this case, we can make the deterministic variable r_t into a random variable $R(s_t,a)$ that has some underlying probability distribution. If there is randomness in how states evolve into new states, the Q function must be a random variable as well. The original Bellman equation can now be represented as

$$Q(s_t,a_t) \leftarrow \mathbb{E}[R(s_t,a)] + \gamma \cdot \mathbb{E}[Q(S_{t+1},A_{t+1})]$$

Again, the Q function is a random variable because we interpret the environment as having stochastic transitions. Taking an action may not lead to the same next state, so we get a probability distribution over next states and actions. The expected Q value of the next state-action pair is the most likely Q value given the most likely next state-action pair.

If we get rid of the expectation operator, we get a full distributional Bellman equation:

$$Z(s_t,a_t) \leftarrow R(s_t,a_t) + \gamma \cdot Z(S_{t+1},A_{t+1})$$

Here we use Z to denote the distributional Q value function (which we will also refer to as the *value distribution*). When we do Q-learning with the original Bellman equation, our Q function will learn the expected value of the value distribution because that is the best it can do, but in this chapter we will use a slightly more sophisticated neural network that will return a